

Full Stack Engineer — Take-Home Assignment

Property Lead Management API

Role	Full Stack Engineer (Backend-Heavy)
Time	48 hours from receipt (expect ~2–3 hours of focused work)
Submission	GitHub repo link (public or invite shared handle)
Stack	Node.js (NestJS or Express) + TypeScript + PostgreSQL
Cost	Zero. All tools are open source. No paid services or deployments required.

Scenario

You're building the backend for a real estate platform that helps buyers find resale properties. When a property is listed, interested buyers reach out — these are called **leads**. Agents work these leads through a pipeline until a deal closes or the buyer drops off.

Build the API that powers this lead pipeline. No frontend required — Swagger or a Postman collection is your UI.

What to Build

1. Database & Seed Data

Design the schema and write a seed script. Use migrations (Prisma, TypeORM, or Knex).

- **properties** — id, title, address, city, price, bedrooms, status (Available | Booked), created_at
- **leads** — id, buyer_name, phone, email, property_id (FK), status (New | Contacted | Visited | Booked | Lost), priority (Hot | Warm | Cold), created_at, updated_at
- **users** — id, name, email, password_hash, role (Admin | Agent), created_at

Seed with: 10 properties across 2 cities, 15 leads spread across statuses and priorities, 1 admin user and 2 agent users.

2. API Endpoints

Auth

- POST /auth/login — accepts email + password, returns a JWT. No signup needed (users are seeded).
- All other routes require a valid JWT.

Properties (read-only)

- GET /properties — list all properties. Support filtering by city, status, and bedrooms. Paginate with page + limit.

- GET /properties/:id — single property with a count of leads grouped by status.

Leads

- POST /leads — create a lead. Validate: property must exist and be 'Available'; phone and email are required and properly formatted; reject duplicate leads (same phone + same property).
- GET /leads — list leads. Filter by: status, priority, property_id. Paginate.
- GET /leads/:id — single lead with its property details.
- PATCH /leads/:id — update priority or notes only (not status — that uses the transition endpoint).

Lead Status Transitions

This is the core of the assignment. Get this right.

- POST /leads/:id/transition — body: { status }
- Pipeline order: **New** → **Contacted** → **Visited** → **Booked**
- Rules:
 - Leads can only move forward by one step at a time (New → Contacted is OK, New → Visited is not).
 - 'Lost' can be set from any status except 'Booked' — it's an exit at any stage.
 - 'Booked' and 'Lost' are terminal. No further transitions allowed.
 - When a lead moves to 'Booked', automatically set the property's status to 'Booked'.
- Return **clear, specific error messages** on invalid transitions. Example: "Cannot move from 'New' to 'Visited'. Next valid status is 'Contacted'."

Dashboard

- GET /dashboard/summary — return a single JSON response with: properties by status, leads by status, leads by priority, and conversion rate (Booked / Total leads).

3. Tests (3 minimum)

- A valid transition (e.g., New → Contacted) succeeds and updates the lead.
- An invalid transition (e.g., New → Visited) returns the correct error message.
- Creating a duplicate lead (same phone + same property) is rejected.

4. README

Your README should include:

- Setup instructions — how to install dependencies, set up the database, run migrations and seed, and start the server. We should be able to run it locally in under 3 minutes.
- A brief note (3–5 sentences) on one design decision you made and why.
- One thing you'd improve or add if you had more time.

Bonus (Optional — only if the core is done cleanly)

Pick at most one. Both can be done at zero cost:

- **Caching:** Cache the /dashboard/summary response with Redis. Invalidate when a lead is created or transitioned. Document your TTL choice in the README. (Redis can be run locally via Docker — no

paid service needed.)

- **AI Lead Scoring:** When a lead is created, call a free-tier LLM API (Google Gemini free tier, or any other free option) with the lead and property details. Return a suggested priority (Hot / Warm / Cold) with a one-line reason. Include the prompt you used in the codebase.

Evaluation Criteria

Criteria	Weight	What We Look For
Transition Logic & Validations	35%	State machine is correct. Edge cases handled — terminal states, skip prevention, auto-booking on deal close. Clear, helpful error messages.
API Design & Code Quality	25%	RESTful conventions. Controller/service separation. Input validation. Consistent error and response format. Proper HTTP status codes.
Database Design	20%	Sensible schema with proper types, constraints, foreign keys, and indexes on columns you filter by. Working migrations and seed script.
Testing & Documentation	20%	Tests verify transition rules and validation logic. README lets us run the project locally in under 3 minutes.

Submission

- **GitHub repo** (public, or invite the handle shared with you) with a working README.
- **Swagger / Postman collection** included in the repo — we will test your API locally before reading code.

What We Don't Care About

- Frontend. Zero UI needed. Swagger is enough.
- Deployment. We'll run it locally. Don't spend money on hosting.
- Over-engineering. No microservices, no message queues, no event sourcing.
- 100% test coverage. 3 real tests that catch real bugs.
- Fancy auth. JWT with seeded users is plenty.

Show us clean code, working logic, and clear thinking. That's it.

Questions? Reach out to your hiring contact. Good luck!